SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

**CSE - Academic Year 2026-27**
**Java Programming Lab | Batch 2025-29**

# ASSIGNMENT : 26

| | |
|---|---|
| **Name of Student** | Anushree Shinde |
| **PRN No.** | 25070122041 |
| **Batch** | CSE A - 2 |
| **Class** | Java Programming Lab |
| **Academic Year & Semester** | 2025-29 : Semester 3 |
| **Date of Submission** | |

**TITLE :** Write a Java program to create and execute multiple threads using the Runnable interface

**Learning Objectives :** To create multiple threads using the Runnable interface.

**Theory :**

The Runnable interface defines the run() method containing the task performed by a thread. An object implementing Runnable is passed to a Thread object, and start() begins execution. This separates task logic from thread control and allows the class to extend another superclass.

---

**Exercise :**

1. Create a program using Runnable to run two tasks simultaneously.

```java
class PrintTask implements Runnable {
    String name;

    PrintTask(String name) {
        this.name = name;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(name + ": " + i);
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        PrintTask p1 = new PrintTask("Task 1");
        PrintTask p2 = new PrintTask("Task 2");

        Thread t1 = new Thread(p1);
        Thread t2 = new Thread(p2);

        t1.start();
        t2.start();
    }
}
```

OUTPUT :

```
PS C:\Users\softlink\Desktop\java code>  c:; cd 'c:\Users\
nk\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.
tlink\AppData\Local\Temp\cp_4t8lz63b99atog5tgddg57eiz1.arg
Task 2: 4
Task 2: 5
Task 1: 3
Task 1: 4
Task 1: 5
PS C:\Users\softlink\Desktop\java code>
```

2. Use Runnable for concurrent deposit and withdrawal operations.

```
class BankAccount {
    int balance = 1000;
    synchronized void deposit(int amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}

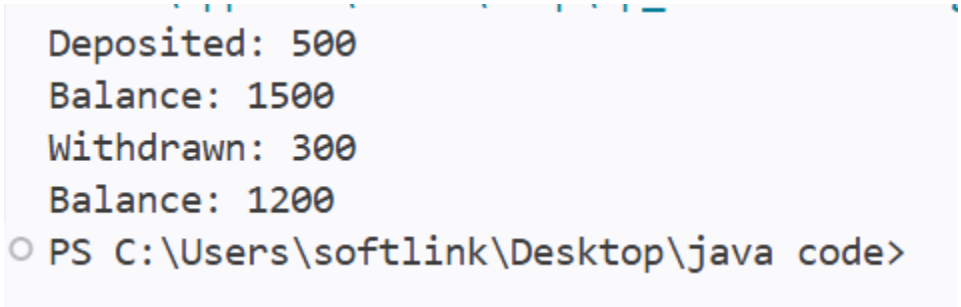
class DepositTask implements Runnable {
    BankAccount account;
    DepositTask(BankAccount account) {
        this.account = account;
    }
    public void run() {
        account.deposit(500);
    }
}

class WithdrawTask implements Runnable {
    BankAccount account;
    WithdrawTask(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.withdraw(300);
```

```
    }
}
public class RunnableBank {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        Thread t1 = new Thread(new DepositTask(account));
        Thread t2 = new Thread(new WithdrawTask(account));
        t1.start();
        t2.start();
    }
}
```

OUTPUT :




**Outcome :**

- Understood the use of the Runnable interface for creating threads in Java.
- Learned to implement the run() method to define tasks executed by threads.
- Understood how to create and start threads using the Thread class and start() method.
- Learned to execute multiple tasks concurrently using Runnable.
- Successfully implemented programs for simultaneous task execution and bank deposit and withdrawal operations using Runnable.

**Conclusion :**

This assignment demonstrated how to create and execute multiple threads using the Runnable interface. Concurrent tasks, including number printing and bank transactions, were implemented to understand multithreaded execution. The assignment helped in understanding the separation of task definition from thread execution and the use of synchronization for shared resources.